

Les scénarios

S1 — Achat billet (trajet direct)

Objectif

Permettre à un voyageur d'acheter un billet pour un trajet direct entre deux villes.

Logique détaillée

1. Le voyageur recherche un trajet (Ville A → Ville B).
2. Le système vérifie en base s'il existe un segment direct.
3. Le système affiche les horaires disponibles.
4. Le voyageur sélectionne un horaire et confirme l'achat (paiement simulé).
5. Le système :
 - Génère un **UUID unique**
 - Crée un billet avec état = **VALIDE**
 - Associe le billet au segment
6. Le billet est enregistré en base.
7. Un QR Code contenant uniquement l'UUID est généré.
8. Le voyageur reçoit son billet numérique.

Résultat attendu

Billet créé, traçable, prêt à être contrôlé.

S2 — Achat billet (trajet avec correspondance)

Objectif

Proposer un itinéraire multi-segments cohérent et générer un billet unique.

Logique détaillée

1. Le voyageur recherche un trajet A → C.
2. Le système constate qu'il n'existe pas de segment direct.

3. Il cherche une combinaison possible (ex : $A \rightarrow B + B \rightarrow C$).
4. Il vérifie :
 - Compatibilité des horaires
 - Temps de correspondance suffisant
5. Il propose l'itinéraire complet.
6. Le voyageur confirme.
7. Le système :
 - Génère **un seul UUID**
 - Associe plusieurs segments au billet
 - État = **VALIDE**
8. Génération d'un QR unique.

Résultat

Un billet unique couvrant tous les segments.

S2b — Aucun itinéraire disponible

Objectif

Informer l'utilisateur qu'aucune solution n'existe.

Logique

- Recherche segments directs.
- Recherche correspondances possibles.
- Aucune combinaison valide trouvée.
- Message : "Aucun itinéraire disponible".

S3 — Contrôle d'un billet valide (cas normal)

Objectif

Valider un billet correctement lors d'un contrôle.

Logique détaillée

1. L'agent scanne le QR.
2. L'application envoie :
 - UUID
 - Token d'authentification
 - Numéro train
 - Date/heure
3. Le serveur vérifie :
 - Token valide ?
 - UUID existant ?
 - Billet état = VALIDE ?
 - Segment correspond au train ?
 - Date et heure correctes ?
4. Si tout est correct :
 - Enregistrement d'une trace
 - Segment marqué comme utilisé
5. Réponse : "Billet valide".

S4 — Fraude : double scan

Objectif

Empêcher la réutilisation du même billet.

Logique

1. Un billet est déjà validé sur un segment.
2. Un deuxième agent scanne le même billet.
3. Le système détecte que le segment est déjà validé.
4. Refus immédiat.
5. Trace enregistrée.

S5 — Billet hors itinéraire

Objectif

Refuser un billet utilisé sur un mauvais segment.

Logique

- Le billet contient segments $A \rightarrow B \rightarrow C$.
- L'agent contrôle sur un train $D \rightarrow E$.
- Le serveur vérifie la correspondance segment/train.
- Si le segment ne fait pas partie du billet $\rightarrow$ refus.

S6 — Mauvaise date / heure / train

Objectif

Refuser une validation hors contexte.

Logique

Même si le billet existe :

- Mauvais numéro de train ?
- Mauvaise date ?
- Heure incohérente ?

Refus de validation.

S7 — Billet invalide ou déjà utilisé (état global)

Objectif

Refus automatique si état $\neq$ VALIDE.

Logique

Si état =

- UTILISÉ
- INVALIDE

Refus immédiat sans autre vérification.

S8 — Validations concurrentes

Objectif

Gérer deux scans en même temps.

Logique

- Deux agents scannent simultanément.
- Le serveur traite avec transaction.
- Première validation acceptée.
- Seconde refusée.
- Cohérence garantie en base.

S9 — Billet introuvable

Objectif

Refuser UUID inconnu.

Logique

- UUID reçu.
- Aucune correspondance en base.
- Réponse : "Billet introuvable".
- Trace enregistrée.

S10 — QR illisible / UUID invalide

Objectif

Refuser si format incorrect.

Logique

- UUID mal formé.

- Format invalide.
- Refus avant requête base.

S11 — Token absent / agent non autorisé

Objectif

Sécuriser accès API.

Logique

- Requête sans token.
- Token invalide.
- Refus immédiat.
- Aucun traitement du billet.

S12 — Token expiré

Objectif

Refuser session expirée.

Logique

- Token analysé.
- Date d'expiration dépassée.
- Refus + message "Session expirée".

S13 — Segment bon mais hors fenêtre

Objectif

Empêcher validation trop tôt ou trop tard.

Logique

- Le segment est correct.
- Mais heure actuelle hors plage autorisée.
- Refus de validation.

S14 — Admin consulte historique

Objectif

Assurer traçabilité complète.

Logique

1. Admin saisit UUID.
2. Le système récupère :
 - Date création
 - Segments
 - Validations
 - Refus
3. Affichage historique complet.

S15 — Admin ajoute ville et segment

Objectif

Gérer le réseau ferroviaire.

Logique

1. Ajout nouvelle ville.
2. Ajout segment :
 - Ville départ
 - Ville arrivée
 - Train
 - Horaire
3. Enregistrement base.

S16 — Validation billet avec correspondance

Objectif

Valider progressivement un billet multi-segments.

Logique

Billet $A \rightarrow B \rightarrow C$

- Segment 1 validé $\rightarrow$ marqué utilisé.
- Segment 2 validé $\rightarrow$ marqué utilisé.
- Quand tous segments sont utilisés :
 - État global du billet = UTILISÉ.